KARACHI INSTITUTE OF ECONOMICS & TECHNOLOGY

MOBILE APPLICATION DEVELOPMENT
Project REPORT

TITLE:
Team Fortress Defence – Side-Scroller Survival & Tower Defence

SUBMITTED BY:
Maaz Habib Khan – 65865
Hamza Imran - 15764
Asad Rehman – 65878

SUBMITTED TO:
Miss Erum Aman

Table of Contents

1. Scene 1: Main Menu Screen (`MainMenu.unity`)
2. Scene 2: Core Gameplay / Battle Screen (`Stage1.unity`)
3. Scene 3: Game Over Screen (`GameOver.unity`)
4. Scene 4: Victory / Win Screen (`WinScene.unity`)
5. Overall Architectural Flow & Script Interaction

Scene 1: Main Menu Screen (`MainMenu.unity`)

1. Scene Overview & Description

The **Main Menu** serves as the initial entry point of the game application. Its primary purpose is to welcome the player, establish the thematic aesthetic of the game (a zombie survival/apocalypse theme), and provide clean navigation paths to start the game or exit the application.

2. User Interface (UI) Components & Layout

The interface is designed with a minimalistic, sleek dark theme featuring high-contrast text and buttons:

- **Game Title Header:** A prominent TextMeshPro (TMP) header showing the name of the game (e.g., *Zombie Outbreak* or *Undead Vanguard*) in a stylised horror/survival font.
- **Play Button:** A stylized interactive button that allows players to launch the game.
- **Quit Button:** An interactive button allowing players to exit the game application.
- **Background Panel:** An atmospheric background sprite or gradient that sets the dark, apocalyptic tone of the game.

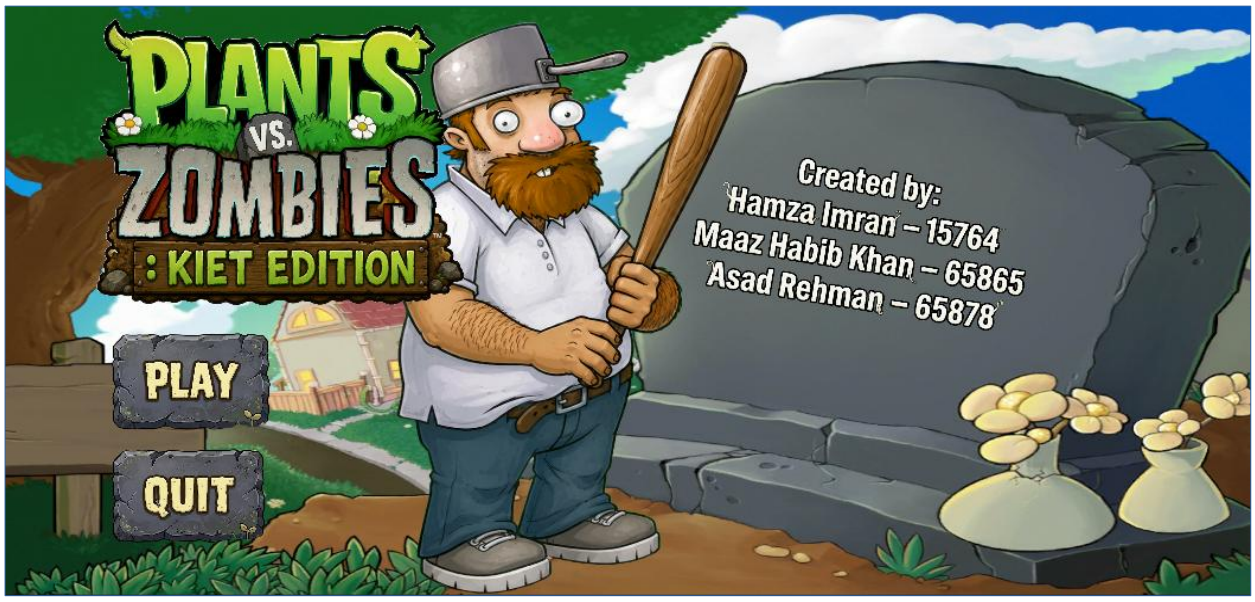
3. Interactive Mechanics & Navigation

- **Starting the Game:** Clicking or tapping the **Play** button immediately initiates a transition, loading the first level (`Stage1`).
- **Exiting the App:** Clicking or tapping the **Quit** button exits the application (closable in standalone builds).

4. Technical Script Integration

This scene is driven by the custom `**SceneLoader.cs**` script attached to a UI Manager or Event System object:

- **Play Button Functionality:** Triggers `SceneLoader.LoadScene("Stage1")`, which uses Unity's `UnityEngine.SceneManagement` to load the gameplay scene.
- **Quit Button Functionality:** Triggers `SceneLoader.QuitGame()`, executing `Application.Quit()` to safely shut down the game runtime.



Scene 2: Core Gameplay / Battle Screen (`Stage1.unity`)

1. Scene Overview & Description

The **Stage 1 Scene** contains the core tactical gameplay loop of the game. The player controls a Hero character horizontally across a 2D battlefield and must defend against incoming waves of zombies. The player has the unique ability to summon AI-controlled allies to help fight the undead. Victory is achieved by eliminating all spawned zombies, while defeat occurs if the player's health drops to zero.

2. User Interface (UI) / Heads-Up Display (HUD)

Designed for mobile-friendly touch layouts (Mobile Application Development standard), the HUD overlay includes:

- **Horizontal Movement Buttons:**
  - `Move Left` Arrow Button (on-screen touch button).
  - `Move Right` Arrow Button (on-screen touch button).
- **Action Buttons:**
  - `Attack` Button: Triggers the player's physical swing/slashing attack.
  - `Spawn Ally 1` Button: Summons the first type of AI ally (with visible cooldown).
  - `Spawn Ally 2` Button: Summons the second type of AI ally (with visible cooldown).
- **Dynamic UI Elements:** Health indicators, cooldown overlays, and spawner UI.

3. Interactive Gameplay Mechanics

- **Player Control:** The player uses horizontal touch inputs to navigate. The hero's sprites swap dynamically between running/idle animations and a dedicated slashing frame during attacks.
- **Combat System:** When the `Attack` button is pressed, the player checks a circular overlap area (`attackPoint`) and deals damage to all objects tagged as "Enemy" within range. Hits trigger dynamic particle effects (`hitEffect`) and apply a physical knockback force.
- **Wave System:** An automated `EnemySpawner` coordinates zombie spawning. After a 3-second delay, it releases a wave of 10 zombies at a designated spawn point at 2-second intervals.
- **Ally Summoning System:** Using the on-screen buttons, the player can summon two distinct types of defenders at corresponding spawn points (`AllySpawner`). Each type has its own cooldown timer to prevent spamming.

- **Zombie AI (Enemy):** Zombies scan the arena. They prioritize targeting any spawned **Allies** first. If no allies are active, they lock onto the **Player**. They move towards their target and automatically deal damage in close range.
- **Ally AI (Defender):** Summoned allies automatically scan for the closest active enemy within a detection radius of 6 units. They move to engage the target and attack them automatically.

4. Technical Script Integration

This scene utilizes the core of the game's codebase:

- **`PlayerController.cs`**: Handles horizontal movement via `MoveLeft()`, `MoveRight()`, and `StopMoving()` functions hooked to the UI button pointer triggers.
- **`PlayerAttack.cs`**: Performs the overlap circle physics check and applies damage. Swaps sprites using `PlayerSpriteSwap.cs` and instantiates damage visual effects.
- **`ZombieAI.cs`**: Governs zombie target prioritization, movement, and attacking logic. Registers/deregisters with the central game loop controller.
- **`AllyAI.cs`**: Handles ally target scanning (finding the closest object tagged "Enemy") and automated combat.
- **`AllySpawner.cs`**: Spawns ally prefabs at coordinates and maintains the cooldown timers.
- **`EnemySpawner.cs`**: Manages the timed coroutine to spawn waves of zombies.
- **`Health.cs`**: Tracks individual HP for the player, allies, and zombies. Applies physics-based knockback using `Rigidbody2D.AddForce` and invokes destruction/death events.
- **`GameManager.cs`**: Tracks the overall count of active enemies. Coordinates transitions to `WinScene` (when enemy count reaches 0) or `GameOver` (when the Player dies).
- **`MusicManager.cs`**: Plays looping, atmospheric survival music.



Scene 3: Game Over Screen (`GameOver.unity`)

1. Scene Overview & Description

The **Game Over Scene** is triggered automatically when the player's health reaches zero. It functions as a feedback screen, informing the player of their defeat, halting the gameplay loops, and providing interactive options to quickly restart the stage or return to the main hub.

2. User Interface (UI) Components & Layout

- **Defeat Header:** A large, high-visibility title text displaying "GAME OVER" or "DEFEATED" in blood-red typography.
- **Try Again Button:** A primary interactive button that reloads the core gameplay level.
- **Main Menu Button:** A secondary button that returns the player to the introductory menu screen.
- **Aesthetic Panel:** A dark, moody background panel emphasizing the defeat state.

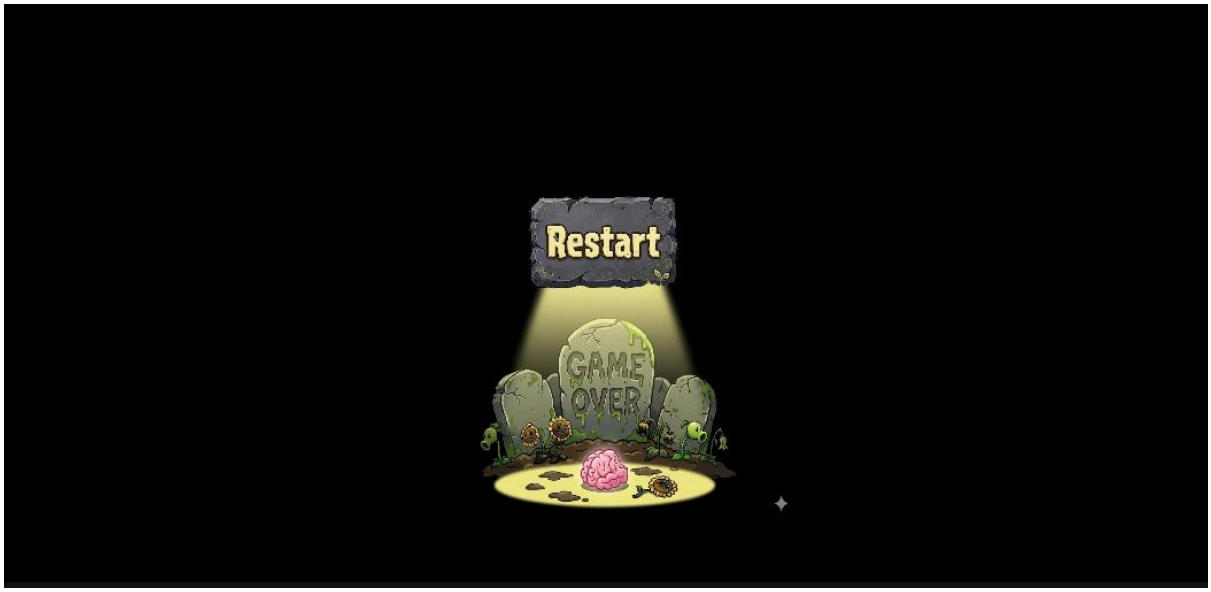
3. Interactive Mechanics & Navigation

- **Restarting Stage 1:** Tapping **Try Again** immediately wipes the previous game state and restarts the battle scene from the beginning.
- **Returning to Menu:** Tapping **Main Menu** returns the user to the opening menu.

4. Technical Script Integration

This scene utilizes the `**SceneLoader.cs**` script linked to the UI buttons:

- **Try Again Button:** Triggers `SceneLoader.LoadScene("Stage1")`, resetting the level and restarting all wave spawner coroutines.
- **Main Menu Button:** Triggers `SceneLoader.LoadScene("MainMenu")`, returning the player to the initial screen.
- **State Resetting:** The central `GameManager` instance is destroyed or reset to ensure a fresh, clean gameplay state.



Scene 4: Victory / Win Screen (`WinScene.unity`)

1. Scene Overview & Description

The **Victory Screen** represents the successful completion of the game's main level. It is loaded when the player and their allies successfully defeat the entire zombie horde spawned by the wave controller. It rewards the player with positive feedback and provides options to play again or navigate back to the main menu.

2. User Interface (UI) Components & Layout

- **Victory Header:** A bright, bold congratulatory title displaying "VICTORY!", "YOU WIN!", or "STAGE CLEARED" in golden or high-contrast heroic typography.
- **Play Again Button:** A prominent interactive button allowing players to replay the stage.
- **Main Menu Button:** A button allowing players to navigate back to the initial start screen.
- **Trophy/Success Graphic:** Visual icons or sprites celebrating the accomplishment.

3. Interactive Mechanics & Navigation

- **Replaying:** Tapping **Play Again** restarts `Stage1` for another round of gameplay.
- **Exiting to Menu:** Tapping **Main Menu** takes the player back to the main menu screen.

4. Technical Script Integration

This scene is driven by the `**SceneLoader.cs**` script attached to the victory UI layout:

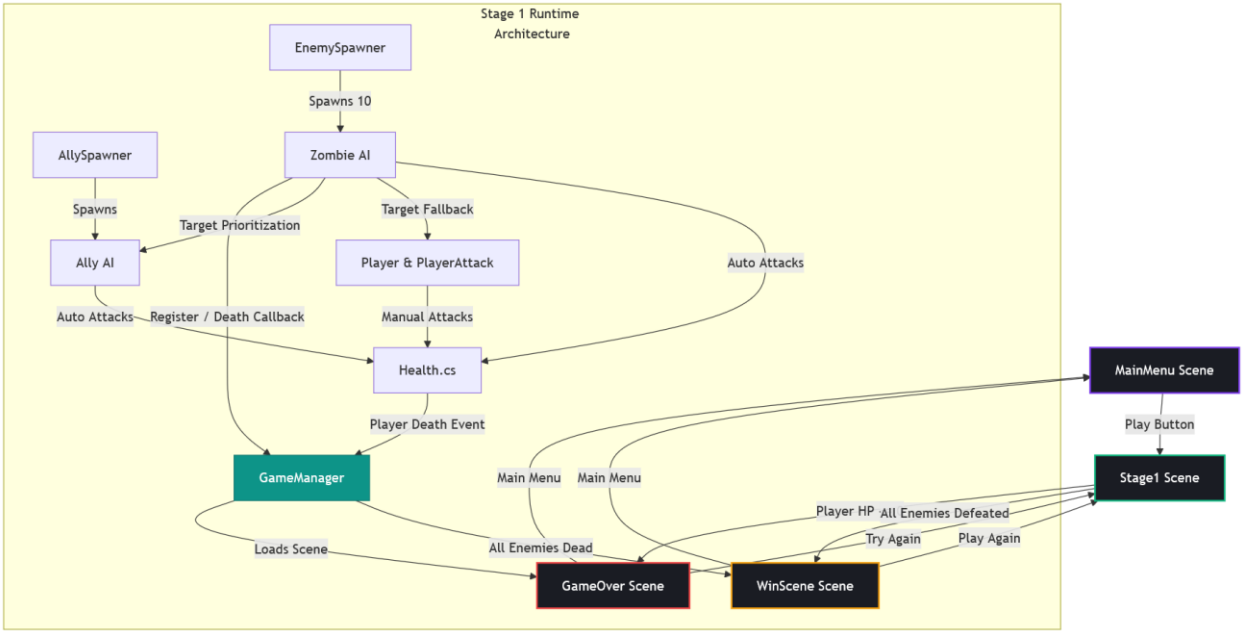
- **Play Again Button:** Triggers `SceneLoader.LoadScene("Stage1")`.

- **Main Menu Button:** Triggers `SceneLoader.LoadScene("MainMenu")`.



Overall Architectural Flow & Script Interaction

This diagram represents how the scenes and their core C# scripts communicate dynamically during game runtime.



Script Execution Summary

1. **Scene Transitioning:** Handled globally by `SceneLoader.cs` which leverages the standard Unity `SceneManager` API.

2. **Gameplay State Management:** Orchestrated by `GameManager.cs` (a persistent Singleton). It manages register/unregister counters for enemies.

3. **Player Movement & Attacks:** Controlled by `PlayerController.cs` and `PlayerAttack.cs` which are triggered directly via UI Event Triggers hooked up to touch canvas buttons.

4. **Enemy & Ally Artificial Intelligence:** Decoupled. `ZombieAI.cs` targets `Ally` tags first and falls back to `Player` tags, while `AllyAI.cs` targets `Enemy` tags, creating a self-regulating automated combat grid.

5. **Universal Health System:** The `Health.cs` script is attached to every combatant. It deducts health, applies visual damage responses, triggers knockback on `Rigidbody2D`, and handles cleanup/callbacks upon reaching `0` HP.